# linear#

https://pytorch.org/docs/stable/generated/torch.ao.nn.quantized.functional.l

**Description:**

Applies a linear transformation to the incoming quantized data: y=xAT+by = xA^T + by=xAT+b. See Linear Current implementation packs weights on every call, which has penalty on performance. If you want to avoid the overhead, use Linear. input (Tensor) – Quantized input of type torch.quint8 weight (Tensor) – Quantized weight of type torch.qint8

## Function Signature

```
class torch.ao.nn.quantized.functional.linear(input, weight, bias=None, scale=None, zero_point=None)[source]#
Parameters
Return type
Shape:
```

**Returns:**

Tensor

## Notes & Warnings

NOTE Note Current implementation packs weights on every call, which has penalty on performance. If you want to avoid the overhead, use Linear.

# Detailed Documentation

## Documentation

Applies a linear transformation to the incoming quantized data: y=xAT+by = xA^T + by=xAT+b. See Linear

Current implementation packs weights on every call, which has penalty on performance. If you want to avoid the overhead, use Linear.

input (Tensor) – Quantized input of type torch.quint8

weight (Tensor) – Quantized weight of type torch.qint8

bias (Tensor) – None or fp32 bias of type torch.float

scale (double) – output scale. If None, derived from the input scale

zero_point (python:long) – output zero point. If None, derived from the input zero_point

Input: (N,∗,in_features)(N, *, in_features)(N,∗,in_features) where * means any number of additional dimensions

Weight: (out_features,in_features)(out_features, in_features)(out_features,in_features)

Bias: (out_features)(out_features)(out_features)

Output: (N,∗,out_features)(N, *, out_features)(N,∗,out_features)